Yeti-Travel Group Project

Context

Yeti Travel is a travel agency specialised in **school trips**. Over the past year the company has registered a concerning **drop in sales, especially among repeat customers**. Since most of the revenue comes from loyal clients, this threatens the continuity of the business.

Our goal is to:

1. Build a predictive model that estimates the probability that a client (a school) will come back the following year (`Retained = 1`) or churn (`Retained = 0`).
2. Identify the most informative features of loyal customers, so that the agency can design targeted retention strategies (personalised offers, tighter relationships with high-risk schools, etc.).

Data

The data is split across three departments, each with its own snapshot at the end of "year 1":

- Sales: trip-level commercial info (program, grades, travel mode, dates, passengers). It contains the target `Retained` .
- Finance: financial info (tuition, insurance take-up, revenue, payment modality).
- CRM: school-level info (school type, size, region, parent meetings, income level).

1. Setup and imports

We use the standard data-science stack: `pandas` / `numpy` for data handling, `matplotlib` / `seaborn` for visualisation, and `scikit-learn` for the full ML pipeline (preprocessing, cross-validation, models).

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn

from sklearn.model_selection import StratifiedKFold, cross_validate
```

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoosti
from sklearn.metrics import roc_auc_score, f1_score, classification

import warnings
warnings.filterwarnings('ignore')

sns.set_style('whitegrid')
plt.rcParams['figure.dpi'] = 100
```

2. Loading the data

We load both the **training (`_model`)** and the **test (`_test`)** versions of the three datasets. The training set is labelled (`Retained` is known) and will be used to fit the model. The test set is unlabelled — our model's predictions on it are the deliverable.

We store every dataframe in a single dictionary so the loading logic stays DRY and we can loop over them for a quick shape check.

In [ ]:
```
# set the path to our data folder
path = 'data/'
files = ['sales', 'finance', 'crm']

# initialize dictionary of datasets
data = {}
for name in files:
    data[f'{name}_train'] = pd.read_csv(f'{path}{name}_model.csv')
    data[f'{name}_test'] = pd.read_csv(f'{path}{name}_test.csv')

# For each of the 6 dataframes, print a summary
for key, df in data.items():
    # separator made by = sign
    print(f"\n{'='*40}")
    # print the specific df and the corresponding shape
    print(f"{key}: {df.shape}")
    # print first 3 rows
    print(df.head(3).to_string())
    # summary of missing values (only for those with more than 0 mi
    missing = df.isnull().sum()
    missing = missing[missing > 0]
    if not missing.empty:
        print("\nMissing values:")
        print(missing)
    else:
        print("\nNo missing values.")
```

```
========================================
sales_train: (4153, 24)
```

```
  ID_SALES Program_Code  From_Grade  To_Grade Group_State  Days Trav
el_Type Departure_Date Return_Date   Early_RPL  Latest_RPL  Cancelle
d_Pax  Total_Discount_Pax Initial_System_Date SPR_Product_Type  FPP
Total_Pax DepartureMonth GroupGradeTypeLow GroupGradeTypeHigh GroupG
radeType MajorProgramCode FPP_to_School_enrollment  Retained
0  CC1387A           CC        10.0      10.0          CA    24
A     04/07/2019  05/01/2019  05/11/2018  04/10/2018             2
1          04/01/2018       East Coast   26         29        Januar
y                 K               High        K->High
C        0,126429354314411         1
1   CC139A           CC         5.0       6.0          CA    18
A     04/30/2019  05/18/2019  03/05/2018  03/06/2018             2
3          03/16/2018       East Coast   21         22        Januar
y                PK             Middle     PK->Middle
C        0,173405156631824         0
2  CC1600A           CC         7.0       7.0          OR    13
A     03/17/2019  03/30/2019         NaN         NaN             0
1          08/03/2018       East Coast    7          7        Januar
y                 K             Middle      K->Middle
C        0,108985714542943         1

Missing values:
From_Grade                   226
To_Grade                     269
Early_RPL                   1168
Latest_RPL                    37
Initial_System_Date           16
FPP_to_School_enrollment     159
dtype: int64

========================================
sales_test: (630, 24)
  ID_SALES Program_Code  From_Grade  To_Grade Group_State  Days Trav
el_Type Departure_Date Return_Date   Early_RPL  Latest_RPL  Cancelle
d_Pax  Total_Discount_Pax Initial_System_Date SPR_Product_Type  FPP
Total_Pax DepartureMonth GroupGradeTypeLow GroupGradeTypeHigh GroupG
radeType MajorProgramCode  FPP_to_School_enrollment Unnamed: 23
0  CC1040A           CC         7.0       7.0          AZ    25
A     01/29/2019  02/23/2019  02/27/2018  02/04/2018             0
5          01/26/2018       East Coast   60         60       Decembe
r            Middle               High   Middle->High
C                  0.103591         NaN
1  CC1525A           CC        10.0      10.0          CA    22
A     05/03/2019  05/25/2019  02/08/2018  08/11/2018            12
10          01/31/2018       East Coast   70         79        Janua
ry              High               High     High->High
C                  0.101837         NaN
2  CC2011A           CC         8.0      10.0          TX     9
A     05/08/2019  05/17/2019  02/20/2018  02/16/2018             3
1          03/18/2018       East Coast    9          9        Januar
y            Middle               High   Middle->High
C                  0.036809         NaN

Missing values:
From_Grade                    28
To_Grade                      31
```

```
Early_RPL                   179
Latest_RPL                    2
FPP_to_School_enrollment     23
Unnamed: 23                 629
dtype: int64

========================================
finance_train: (4151, 12)
  ID_FINANCE Deposit_Date Special_Pay  Tuition  FRP_Active  FRP_Canc
elled FRP_Take_up_percent_ EZ_Pay_Take_Up_Rate  School_Sponsor  SPR_
Group_Revenue         FPP_to_PAX  Num_of_Non_FPP_PAX
0       1389   11/08/2018           0      397           1
1    0,020913435758935   0,017855577091883              1
447  0,967035782753087                   2
1       3090   01/06/2019           0      563           0
0                    0                   0              0
579  0,993127324453424                   1
2       3639   12/29/2018           0      617           0
0                    0                   0              0
529  0,973945817054653                   1

Missing values:
Special_Pay    3322
dtype: int64

========================================
finance_test: (630, 12)
  ID_FINANCE Deposit_Date Special_Pay  Tuition  FRP_Active  FRP_Canc
elled  FRP_Take_up_percent_  EZ_Pay_Take_Up_Rate  School_Sponsor  SP
R_Group_Revenue  FPP_to_PAX  Num_of_Non_FPP_PAX
0     CP1005   11/20/2018          CP      424          14
4              0.641553             0.043442               1
439    0.972280                   1
1     CP1040   05/22/2018          CP     2008          32
0              0.564888             0.265647               0
2175    0.913557                   4
2     CP1261   10/20/2018          CP     1699          28
3              0.793570             0.148866               0
1798    0.909598                   4

Missing values:
Special_Pay    514
dtype: int64

========================================
crm_train: (4148, 20)
   ID_CRM Poverty_Code Region  CRM_Segment School_Type  Parent_Meeti
ng_Flag MDR_Low_Grade  MDR_High_Grade  Total_School_Enrollment Incom
e_Level  SPR_New_Existing  NumberOfMeetingswithParents FirstMeeting
LastMeeting  DifferenceTraveltoFirstMeeting  DifferenceTraveltoLastM
eeting SchoolGradeTypeLow SchoolGradeTypeHigh SchoolGradeType School
SizeIndicator
0  01000I            0  Other         10.0    Catholic
1           PK             8.0                      403
I                 0                           2   05/12/2018  09/1
9/2018                           371.0                          241.
```

```
0             Middle              Middle  Middle->Middle
S-M
1  01001O            0  Other          9.0         CHD
1             K            12.0                      732
O                 0                            1   09/05/2018  09/0
1/2018                            150.0                          154.
0             Middle              Middle  Middle->Middle
M-L
2  01002G            0  Other          5.0    Catholic
1             K             8.0                      521
G                 1                            1   09/09/2018  09/0
5/2018                            259.0                          263.
0             Middle              Middle  Middle->Middle
S-M

Missing values:
CRM_Segment                        4
MDR_High_Grade                   116
FirstMeeting                     596
LastMeeting                      596
DifferenceTraveltoFirstMeeting   596
DifferenceTraveltoLastMeeting    596
SchoolSizeIndicator              159
dtype: int64

========================================
crm_test: (630, 20)
   ID_CRM Poverty_Code               Region  CRM_Segment School_Type
Parent_Meeting_Flag MDR_Low_Grade  MDR_High_Grade  Total_School_Enro
llment Income_Level  SPR_New_Existing  NumberOfMeetingswithParents F
irstMeeting LastMeeting  DifferenceTraveltoFirstMeeting  DifferenceT
raveltoLastMeeting SchoolGradeTypeLow SchoolGradeTypeHigh         Sc
hoolGradeType SchoolSizeIndicator
0  01005P            0  Southern California          4.0         CHD
0            PK             6.0                      315
P                 1                            0          NaN
NaN                            NaN                            NaN
Elementary          Elementary  Elementary->Elementary
S
1  01040K            0                Other          5.0         CHD
1             6            12.0                      541
K                 1                            1   09/10/2018  09/0
3/2018                            141.0                          148.
0             Middle              Middle          Middle->Middle
S-M
2  010960            0                Other          7.0      PUBLIC
1             0             NaN                        0
0                 0                            1   09/04/2018  09/0
7/2018                            255.0                          252.
0             Middle              Middle          Middle->Middle
NaN

Missing values:
CRM_Segment                        4
MDR_High_Grade                    20
FirstMeeting                      78
```

```
LastMeeting                        78
DifferenceTraveltoFirstMeeting     78
DifferenceTraveltoLastMeeting      78
SchoolSizeIndicator                23
dtype: int64
```

3. Column configuration and merging

3.1 Column families

Before we touch the data, we declare the column families once at the top. This acts as **single source of truth** for the whole pipeline: if a new column needs special treatment, we add it here and the downstream code picks it up automatically.

In [5]:
```
decimal_comma_cols = [
    'FPP_to_School_enrollment', 'FRP_Take_up_percent_',
    'EZ_Pay_Take_Up_Rate', 'FPP_to_PAX',
]

date_cols = [
    'Departure_Date', 'Return_Date',
    'Early_RPL', 'Latest_RPL',
    'Deposit_Date', 'Initial_System_Date',
    'FirstMeeting', 'LastMeeting',
]

binary_cols = ['School_Sponsor', 'Parent_Meeting_Flag', 'SPR_New_Ex

categorical_cols = [
    'Program_Code', 'Travel_Type', 'Group_State',
    'SPR_Product_Type', 'MajorProgramCode',
    'GroupGradeTypeLow', 'GroupGradeTypeHigh', 'GroupGradeType',
    'DepartureMonth', 'Special_Pay',
    'Poverty_Code', 'Region', 'School_Type', 'Income_Level',
    'SchoolGradeTypeLow', 'SchoolGradeTypeHigh', 'SchoolGradeType',
    'SchoolSizeIndicator', 'CRM_Segment',
]
```

3.2 Merging the three datasets

The three datasets share the same underlying trip, but each department uses its **own ID format**:

Dataset	Example ID
Sales	`CC1040A`
Finance	`CP1040`
CRM	`01040K`

The common element is the **numeric part** (`1040`). We extract it with a regex and use it as the join key.

⚠️ **Important caveat:** if we extract the digits as strings, the CRM IDs preserve their leading zeros (`"01040"` vs `"1040"`) and the join silently fails — producing a merged dataset where all CRM columns are `NaN`. Casting the extracted number to `int` solves it.

We do a **left join from Sales** so we keep every labelled trip even when its CRM row is missing. The `validate='one_to_one'` argument on the first merge asserts that each `trip_id` appears at most once on both sides — if that ever fails, pandas will raise instead of silently producing a broken frame.

In [6]:
```
# create merging function
def merge_datasets(sales, finance, crm):
    sales, finance, crm = sales.copy(), finance.copy(), crm.copy()

    for d in (sales, finance, crm):
        unnamed = [c for c in d.columns if c.startswith('Unnamed')]
        d.drop(columns=unnamed, inplace=True)

    # The regex (\d+) grabs the digit sequence,
    # .str.extract()[0] returns it as a Series, and .astype(int) co
    sales['trip_id']   = sales['ID_SALES'].str.extract(r'(\d+)')[0]
    finance['trip_id'] = finance['ID_FINANCE'].str.extract(r'(\d+)'
    crm['trip_id']     = crm['ID_CRM'].str.extract(r'(\d+)')[0].ast

    # left-join from sales; drop the helper column
    df = (sales
          .merge(finance, on='trip_id', how='left')
          .merge(crm,     on='trip_id', how='left')
          .drop(columns='trip_id'))
    return df
```

In [7]:
```
# run merging function
# create train and test comprehensive df
df_train = merge_datasets(data['sales_train'], data['finance_train'
df_test  = merge_datasets(data['sales_test'],  data['finance_test']

print(df_train.shape, df_test.shape)
df_train.head()
```

```
(4168, 56) (630, 55)
```

Out[7]:

	ID_SALES	Program_Code	From_Grade	To_Grade	Group_State	Days	Tı
0	CC1387A	CC	10.0	10.0	CA	24	
1	CC139A	CC	5.0	6.0	CA	18	
2	CC1600A	CC	7.0	7.0	OR	13	
3	CC1701A	CC	8.0	8.0	CA	24	
4	CC1773A	CC	7.0	7.0	OR	13	

5 rows × 56 columns

4. Preprocessing and validation

Raw merged data is messy: duplicates sneak in from the CRM join, decimals are encoded with commas, dates are strings, and some categorical codes are inconsistent. We fix all of this in a single `preprocess()` function that takes a dataframe and returns a clean one — so it can be applied identically to train and test.

What the function does

1. **Deduplication.** The merge produces duplicate rows on `ID_SALES` because CRM contains multiple variants (a placeholder + a real record) for some trips. We keep the variant with **fewer NaNs** — the most informative one.
2. **Type conversion.** decimals → float, date strings → datetime, `MDR_Low_Grade` string labels (`PK` , `K`) → ordinal integers, binary flags → `Int8` (nullable, memory-efficient), categorical strings → `category` dtype. All conversions are **idempotent** (safe to re-run on an already-converted frame).

In [9]:

```
def preprocess(df):
    """Clean, type-convert, and feature-engineer a merged dataframe
    df = df.copy()

    # Deduplication — keep the most informative row per ID_SALES
    df = (df.assign(_na_count=df.isna().sum(axis=1))
            .sort_values('_na_count', kind='stable')
            .drop_duplicates(subset='ID_SALES', keep='first')
            .drop(columns='_na_count')
            .sort_index()
            .reset_index(drop=True))

    # Fix known anomalies (before type conversion)
    df['Special_Pay'] = df['Special_Pay'].replace('0', np.nan)

    # Decimal commas → float
```

```
    for col in decimal_comma_cols:
        if col in df.columns and not pd.api.types.is_numeric_dtype(
            df[col] = df[col].astype(str).str.replace(',', '.', reg
            df[col] = pd.to_numeric(df[col], errors='coerce')

    # Date strings → datetime
    for col in date_cols:
        if col in df.columns and not pd.api.types.is_datetime64_any
            df[col] = pd.to_datetime(df[col], format='%m/%d/%Y', er

    # MDR grade ordinal mapping
    for col in ['MDR_Low_Grade', 'MDR_High_Grade']:
        if col in df.columns:
            df[col] = pd.to_numeric(
                df[col].replace({'PK': -1, 'K': 0}),
                errors='coerce'
            ).astype('Int8')

    # Binary flags → nullable Int8
    for col in binary_cols:
        if col in df.columns:
            df[col] = df[col].astype('Int8')

    # Categorical strings → 'category' dtype
    for col in categorical_cols:
        if col in df.columns:
            df[col] = df[col].astype('category')

    # Date-derived features (required: sklearn can't handle datetim
    df['booking_lead_days']   = (df['Departure_Date'] - df['Initial
    df['rpl_window_days']     = (df['Latest_RPL']     - df['Early_R
    df['deposit_lead_days']   = (df['Departure_Date'] - df['Deposit
    df['departure_month_num'] = df['Departure_Date'].dt.month.astyp
    df['departure_dow']       = df['Departure_Date'].dt.dayofweek.a

    return df
```

Validation

After preprocessing we run a `validate()` function that asserts the key invariants: unique `ID_SALES`, no fully-empty columns, binary target. If anything breaks silently upstream, this catches it before the model ever sees the data.

In [10]:
```
# validation function
def validate(df, *, expect_target=True):
    """Assert data integrity invariants after preprocessing."""
    assert df['ID_SALES'].is_unique, "ID_SALES must be unique!"
    assert df.columns.is_unique,      "Duplicate column names detec

    all_nan = df.columns[df.isna().all()].tolist()
    assert not all_nan, f"Fully-empty columns: {all_nan}"

    if expect_target:
```

```
        assert 'Retained' in df.columns, "Target column 'Retained'
        assert set(df['Retained'].dropna().unique()) <= {0, 1}, \
            "Retained must be binary!"

    print(f"Validation passed | Shape: {df.shape}")

print('One column difference: Retained column --> our target.')


# Apply to both splits
df_train = preprocess(df_train)
df_test  = preprocess(df_test)

validate(df_train, expect_target=True)
validate(df_test,  expect_target=False)

# quick coverage report
print(f"CRM coverage     — train: {df_train['ID_CRM'].notna().mean(
      f"   test: {df_test['ID_CRM'].notna().mean()*100:5.1f}%")
print(f"Finance coverage — train: {df_train['ID_FINANCE'].notna().m
      f"   test: {df_test['ID_FINANCE'].notna().mean()*100:5.1f}%")
```

```
One column difference: Retained column --> our target.
Validation passed | Shape: (4153, 61)
Validation passed | Shape: (630, 60)
CRM coverage     — train:  97.3%   test:  96.7%
Finance coverage — train:  99.8%   test:  99.8%
```

The validation is checking that 97.3% bookings in the training set have a non-null ID_CRM. This implies they successfully matched to a record in the CRM system. The remaining 2.7% are bookings that exist in sales data but have no CRM record attached. The same holds for the finance coverage report. In general, the coverage is essentially telling you "how much of your merged data is actually complete across all three source systems.

5. Exploratory Data Analysis

With a clean, typed dataframe we can now explore. Three questions drive the EDA:

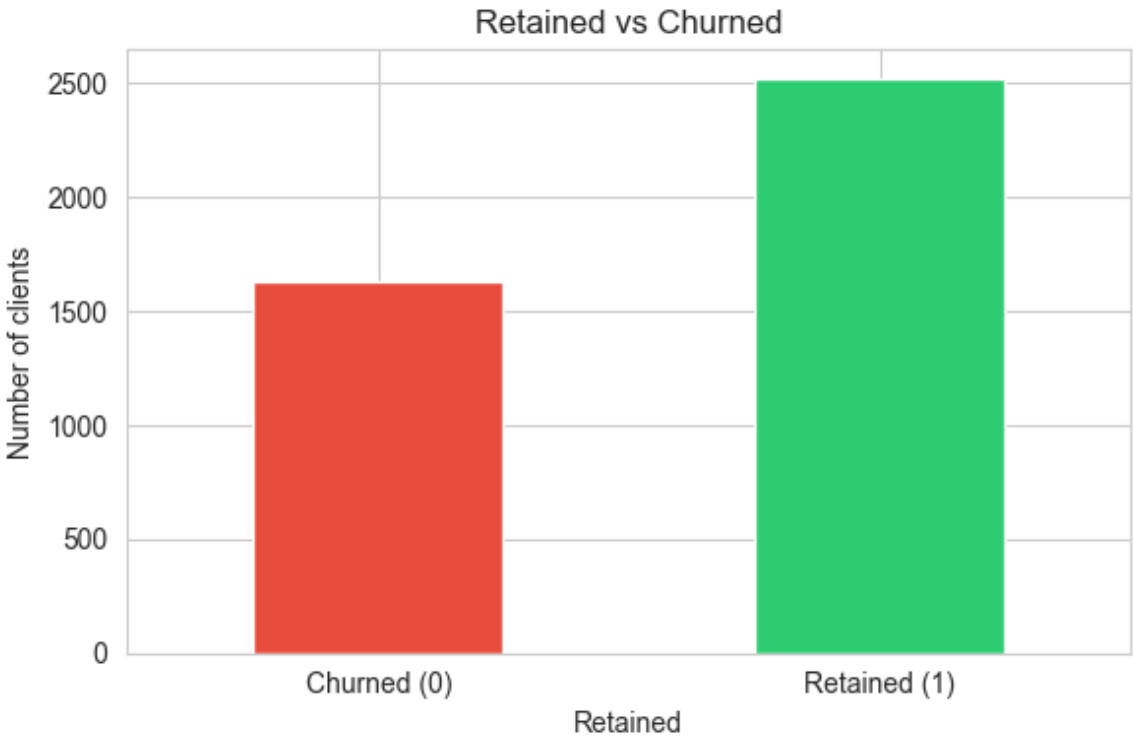
1. **Is the target balanced?** This drives metric choice.
2. **Where are the missing values?** This drives imputation strategy.
3. **Which features look predictive?** This helps feature selection and gives us visual material for the presentation.

5.1 Class balance

If the dataset is heavily imbalanced, accuracy is a misleading metric — we should rely on **ROC-AUC** and **F1** instead.

In [11]:
```
# count Retained target by index (first 0, then 1)
balance     = df_train['Retained'].value_counts().sort_index()
balance_pct = df_train['Retained'].value_counts(normalize=True).sor

print("Counts:\n", balance, sep='')
print("\nProportions:\n", balance_pct, sep='')

# plot
fig, ax = plt.subplots(figsize=(6, 4))
balance.plot(kind='bar', ax=ax, color=['#e74c3c', '#2ecc71'])
ax.set_title('Retained vs Churned')
ax.set_xticklabels(['Churned (0)', 'Retained (1)'], rotation=0)
ax.set_ylabel('Number of clients')
plt.tight_layout()
plt.show()
```

```
Counts:
Retained
0    1631
1    2522
Name: count, dtype: int64

Proportions:
Retained
0    0.393
1    0.607
Name: proportion, dtype: float64
```



Takeaway: the target variable is moderately imbalanced, with 60.7% retained and 39.3% churned clients. This level of imbalance does not require resampling techniques but does rule out accuracy as a primary evaluation metric; we will use ROC-AUC and F1 as main metrics, with the majority-class baseline (60.7%) as our minimum bar for any model.

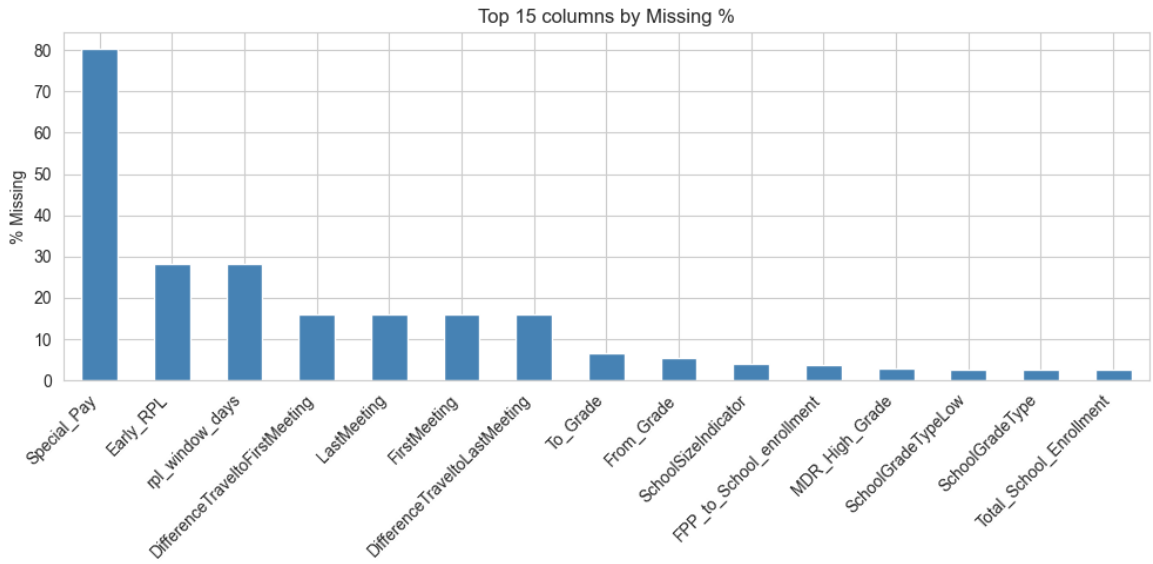
5.2 Missing values

Different columns have very different rates of missingness. A few patterns to notice:

- `Early_RPL` has many nulls — it's the first communication invitation date, likely only recorded when the school reached a certain funnel stage.
- **CRM meeting dates** (`FirstMeeting` , `LastMeeting`) are missing for schools that held no meetings.
- `Special_Pay` is missing for most rows — we'll let imputation handle it but a high missingness rate is itself a signal.

In [12]:
```
# count missing values per column (keep only missing > 0)
missing     = df_train.isnull().sum()
missing     = missing[missing > 0].sort_values(ascending=False)
missing_pct = (missing / len(df_train) * 100).round(1)
print(pd.DataFrame({'Count': missing, 'Percent': missing_pct}).head

# plot
fig, ax = plt.subplots(figsize=(10, 5))
missing_pct.head(15).plot(kind='bar', ax=ax, color='steelblue')
ax.set_title('Top 15 columns by Missing %')
ax.set_ylabel('% Missing')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
                              Count  Percent
Special_Pay                    3329     80.2
Early_RPL                      1168     28.1
rpl_window_days                1168     28.1
DifferenceTraveltoFirstMeeting  661     15.9
LastMeeting                     661     15.9
FirstMeeting                    661     15.9
DifferenceTraveltoLastMeeting   661     15.9
To_Grade                        269      6.5
From_Grade                      226      5.4
SchoolSizeIndicator             169      4.1
FPP_to_School_enrollment        159      3.8
MDR_High_Grade                  126      3.0
SchoolGradeTypeLow              112      2.7
SchoolGradeType                 112      2.7
Total_School_Enrollment         112      2.7
Income_Level                    112      2.7
CRM_Segment                     112      2.7
SPR_New_Existing                112      2.7
MDR_Low_Grade                   112      2.7
School_Type                     112      2.7
```



Takeaway:

- The `Special_Pay` column has about 80% missing values: only four rows have the value '0', and additionally 3,329 out of 4,153 training rows have no value at all.
- The four meeting-related columns (`DifferenceTraveltoFirstMeeting`, `LastMeeting`, `FirstMeeting`, `DifferenceTraveltoLastMeeting`) all have identical missingness (661). This implies that these fields are populated and/or missing as a group, probably because they come from the same system event or data source. If a booking doesn't have a FirstMeeting recorded, it won't have any of the meeting-related fields.
- Eight columns have 112 missing (2.7%): `SchoolGradeTypeLow`, `SchoolGradeType`, `Total_School_Enrollment`, `Income_Level`, `CRM_Segment`, `SPR_New_Existing`, `MDR_Low_Grade`, `School_Type`. This is in line with the previous coverage report: CRM coverage was 97.3%, meaning 2.7% of bookings have no CRM record --> that's exactly 112 rows (2.7% × 4,153 ≈ 112). All eight of these columns come from the CRM table, so they're missing as a block whenever the CRM join fails.

One thing to consider is the `Special_Pay` missingness: it might be important to test whether keeping the missing values, or dropping them, significantly changes the baseline model.

To further confirm our analysis, we can quickly run the same missingness test on the test datasets:

In [13]:
```
missing_test = df_test.isnull().sum()
missing_test = missing_test[missing_test > 0].sort_values(ascending
missing_test_pct = (missing_test / len(df_test) * 100).round(1)
print(pd.DataFrame({'Count': missing_test, 'Percent': missing_test_
```

```
                                Count  Percent
Special_Pay                       514     81.6
rpl_window_days                   179     28.4
Early_RPL                         179     28.4
DifferenceTraveltoFirstMeeting     94     14.9
FirstMeeting                       94     14.9
DifferenceTraveltoLastMeeting      94     14.9
LastMeeting                        94     14.9
To_Grade                           31      4.9
From_Grade                         28      4.4
SchoolSizeIndicator                24      3.8
FPP_to_School_enrollment           23      3.7
SchoolGradeTypeHigh                21      3.3
NumberOfMeetingswithParents        21      3.3
Total_School_Enrollment            21      3.3
CRM_Segment                        21      3.3
MDR_High_Grade                     21      3.3
MDR_Low_Grade                      21      3.3
Parent_Meeting_Flag                21      3.3
School_Type                        21      3.3
Region                             21      3.3
```

5.3 Target rate analysis by categorical variables

This analysis focuses on analyzing the target variable (**`Retained`**) to assess the predictive power of other categorical variables. We are basically computing churn/retain rate for each categorical feature.

Note: we initially considered the `Departure_Month` column, which only took on values "January" and "December", but after a quick check with the initial dataset we understood the presence of a **discrepancy** between the `Departure_Day` column and the actual `Departure_Month`, as presented here:

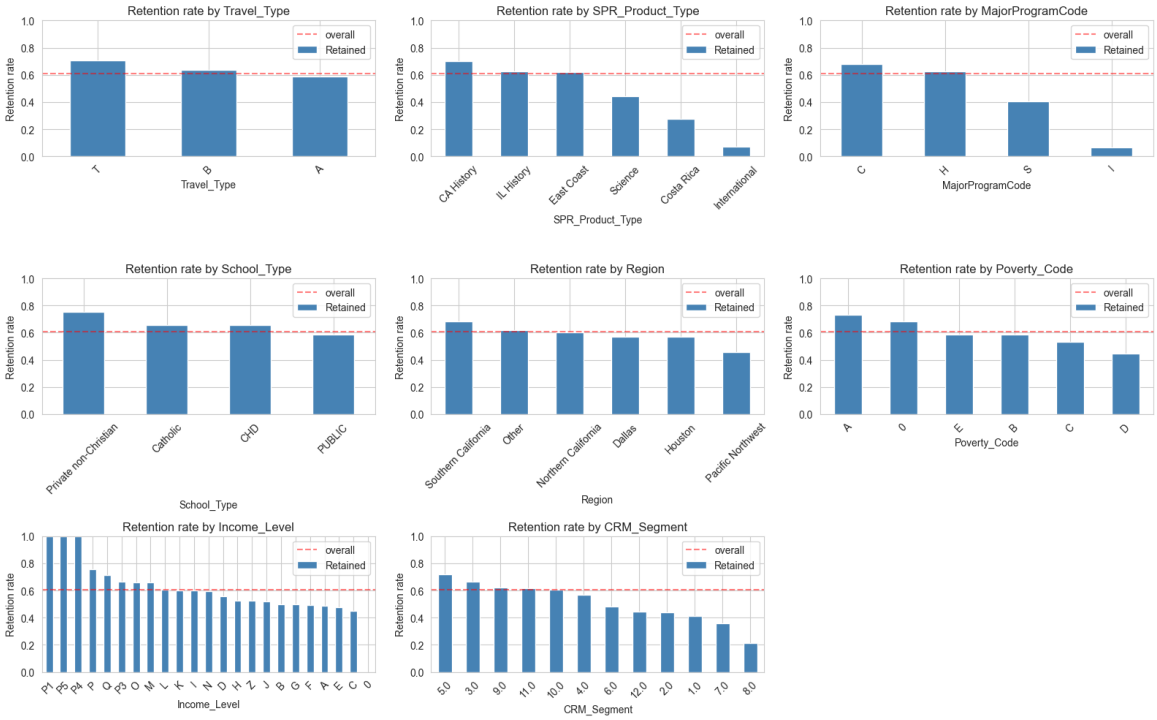
In [14]:
```
# Compare the two columns directly
pd.crosstab(df_train['DepartureMonth'], df_train['departure_month_n

# And check against actual departure dates
print(df_train[['DepartureMonth', 'departure_month_num', 'Departure

# Also check if DepartureMonth might match another date column's mo
print("DepartureMonth vs Initial_System_Date month:")
print(pd.crosstab(
    df_train['DepartureMonth'],
    df_train['Initial_System_Date'].dt.month
))
```

```
   DepartureMonth  departure_month_num Departure_Date
0         January                    4     2019-04-07
1         January                    4     2019-04-30
2         January                    3     2019-03-17
3         January                    5     2019-05-08
4         January                    3     2019-03-17
5         January                    6     2019-06-23
6         January                    3     2019-03-07
7         January                    3     2019-03-09
8         January                    5     2019-05-03
9         January                    4     2019-04-29
10        January                    3     2019-03-30
11        January                    3     2019-03-16
12        January                    3     2019-03-09
13        January                    6     2019-06-16
14        January                    4     2019-04-29
15        January                    5     2019-05-17
16        January                    4     2019-04-27
17        January                    4     2019-04-04
18        January                    5     2019-05-02
19       December                    1     2019-01-29
DepartureMonth vs Initial_System_Date month:
Initial_System_Date  1.0   2.0   3.0   4.0   5.0   6.0   7.0   8.0
9.0   \
DepartureMonth
December               0     4     4     2     1     1     0     0
1
January              345   684   770   728   475   114    82   284
354

Initial_System_Date  10.0  11.0  12.0
DepartureMonth
December               3     0     0
January              163    74    48
```

This highlights that the `Departure_Month` column does not accurately reflect the date information, which is instead correctly presented in `Departure_Date`. Indeed, `DepartureMonth` is documented as 'month of departure' but contains only 'January' (4,137) and 'December' (16), with actual `Departure_Date` values spanning March–June. Interpreted as a program cycle label; seasonal information extracted from `Departure_Date` directly via `departure_month_num`.

In [15]:
```
cat_features_to_check = ['Travel_Type', 'SPR_Product_Type', 'MajorP
                         'School_Type', 'Region', 'Poverty_Code',
                         'Income_Level', 'CRM_Segment']

fig, axes = plt.subplots(3, 3, figsize=(16, 10))
overall_rate = df_train['Retained'].mean()

for ax, col in zip(axes.flat, cat_features_to_check):
    # split by category, compute mean of Retained per bucket
    rates = df_train.groupby(col, observed=True)['Retained'].mean()
    rates.plot(kind='bar', ax=ax, color='steelblue')
```

```
    ax.set_title(f'Retention rate by {col}')
    ax.set_ylabel('Retention rate')
    ax.set_ylim(0, 1)
    # red line at overall retention (60.7%)
    # --> bars above = better than average; below = worse than aver
    ax.axhline(overall_rate, color='red', ls='--', alpha=0.5, label
    ax.legend()
    ax.tick_params(axis='x', rotation=45)

# hide the unused 9th subplot
axes.flat[-1].set_visible(False)
plt.tight_layout()
plt.show()
```



If all bars for a categorical feature sit close to the red line, the feature has little to no predictive power (retention is the same regardless of category). If the bars range from roughly 0.3 to 0.85, the feature is highly predictive. It's also important to check category sizes — if a category only has a handful of rows, the retention rate for that bucket may not be reliable.

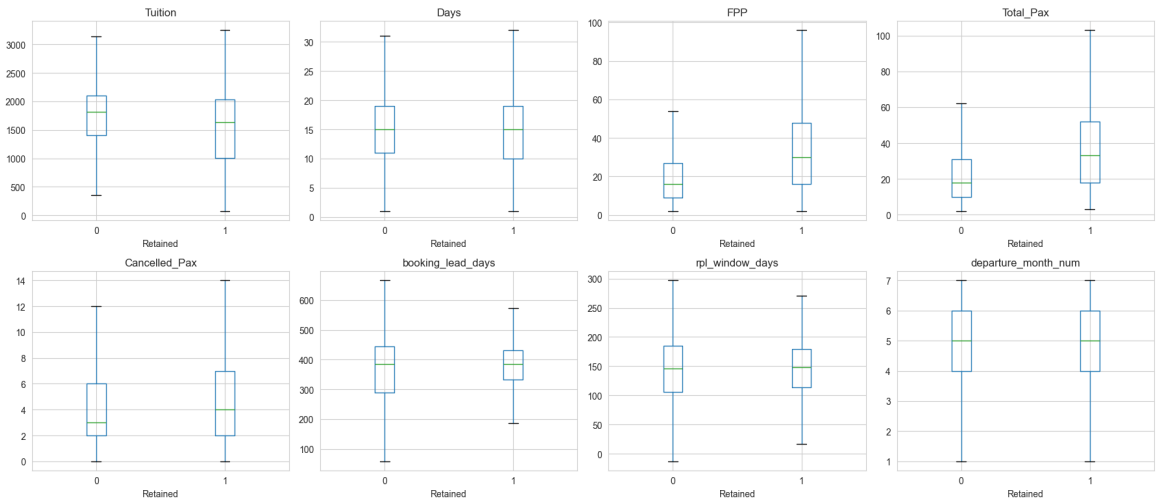
Note: we hand-picked the categorical variables based primarily on readability. Most chosen dimensions have 2 to 5 levels; variables like `Income_Level` have many more categories that would make the plot hard to read.

Takeaway: `SPR_Product_Type`, `MajorProgramCode` and `CRM_Segment` show the widest spread in retention rate across categories — these are likely strong predictors. `Travel_Type` and `Region` show smaller variation and are probably weaker signals.

5.4 Numeric features vs target

For a few key numeric variables we look at their distribution split by target. Different distributions between classes suggest the feature carries signal.

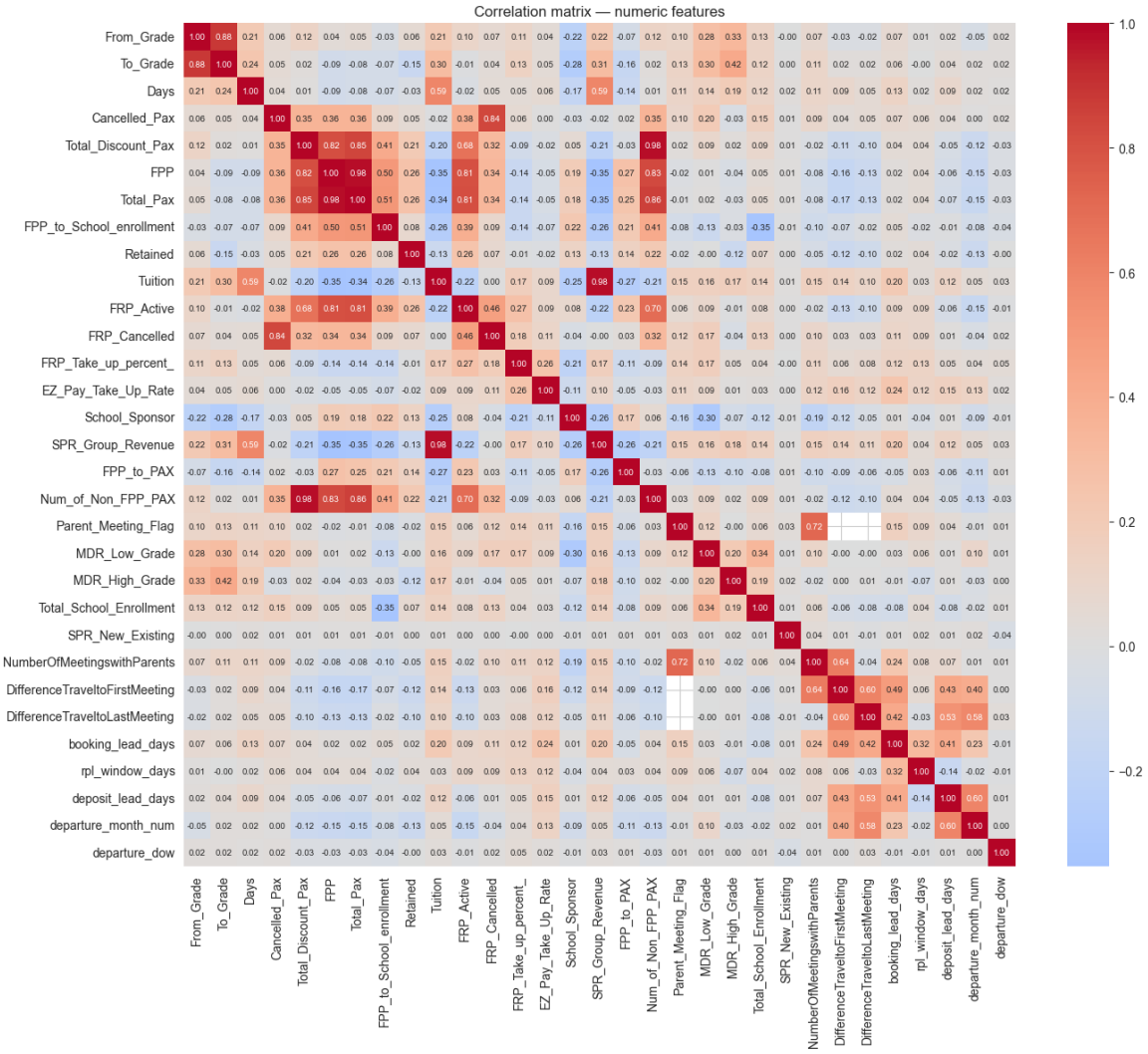
In [16]:
```
num_to_check = ['Tuition', 'Days', 'FPP', 'Total_Pax', 'Cancelled_P
                'booking_lead_days', 'rpl_window_days', 'departure_

fig, axes = plt.subplots(2, 4, figsize=(18, 8))
for ax, col in zip(axes.flat, num_to_check):
    df_train.boxplot(column=col, by='Retained', ax=ax, showfliers =
    ax.set_title(col)
    ax.set_xlabel('Retained')
plt.suptitle('')
plt.tight_layout()
plt.show()
```



5.5 Correlation matrix

Following the approach seen in class on the customer satisfaction dataset, we compute the correlation matrix for all numeric features. This serves two purposes: identifying features most correlated with the target (`Retained`), and spotting multicollinearity between predictors.

In [17]:
```
# correlation matrix for numeric features
num_cols = df_train.select_dtypes(include='number').columns
corr = df_train[num_cols].corr()

fig, ax = plt.subplots(figsize=(14, 12))
sns.heatmap(corr, cmap='coolwarm', center=0, ax=ax,
            annot=True, fmt='.2f', annot_kws={'size': 7})
ax.set_title('Correlation matrix — numeric features')
plt.tight_layout()
plt.show()

# correlation with the target, ranked
print("Correlation with Retained:")
print(corr['Retained'].drop('Retained').sort_values(key=abs, ascend
```


Correlation matrix — numeric features
From_Grade
To_Grade
Days
Cancelled_Pax
Total_Discount_Pax
FPP
Total_Pax
FPP_to_School_enrollment
Retained
Tuition
FRP_Active
FRP_Cancelled
FRP_Take_up_percent_
EZ_Pay_Take_Up_Rate
School_Sponsor
SPR_Group_Revenue
FPP_to_PAX
Num_of_Non_FPP_PAX
Parent_Meeting_Flag
MDR_Low_Grade
MDR_High_Grade
Total_School_Enrollment
SPR_New_Existing
NumberOfMeetingswithParents
DifferenceTraveltoFirstMeeting
DifferenceTraveltoLastMeeting
booking_lead_days
rpl_window_days
deposit_lead_days
departure_month_num
departure_dow
1.0
0.8
0.6
0.4
0.2
0.0
−0.2

```
Correlation with Retained:
FPP                                0.264
Total_Pax                          0.264
FRP_Active                         0.255
Num_of_Non_FPP_PAX                 0.216
Total_Discount_Pax                 0.212
To_Grade                          -0.147
FPP_to_PAX                         0.145
School_Sponsor                     0.128
departure_month_num               -0.127
SPR_Group_Revenue                 -0.126
Tuition                           -0.126
DifferenceTraveltoFirstMeeting    -0.123
MDR_High_Grade                    -0.122
DifferenceTraveltoLastMeeting     -0.097
FPP_to_School_enrollment           0.082
FRP_Cancelled                      0.073
Total_School_Enrollment            0.069
From_Grade                         0.061
NumberOfMeetingswithParents       -0.053
Cancelled_Pax                      0.051
rpl_window_days                    0.036
Days                              -0.035
Parent_Meeting_Flag               -0.024
deposit_lead_days                 -0.021
EZ_Pay_Take_Up_Rate               -0.020
booking_lead_days                  0.018
FRP_Take_up_percent_              -0.007
MDR_Low_Grade                     -0.002
SPR_New_Existing                   0.002
departure_dow                     -0.000
Name: Retained, dtype: float64
```

Takeaway: the features most correlated with `Retained` are the ones to watch most closely in the model. Beyond that, we can spot pairs of highly correlated predictors (e.g. `FPP` and `Total_Pax`, or the various grade columns) — keeping both in a logistic regression would introduce multicollinearity, so in the feature selection step we'll drop the redundant parts.

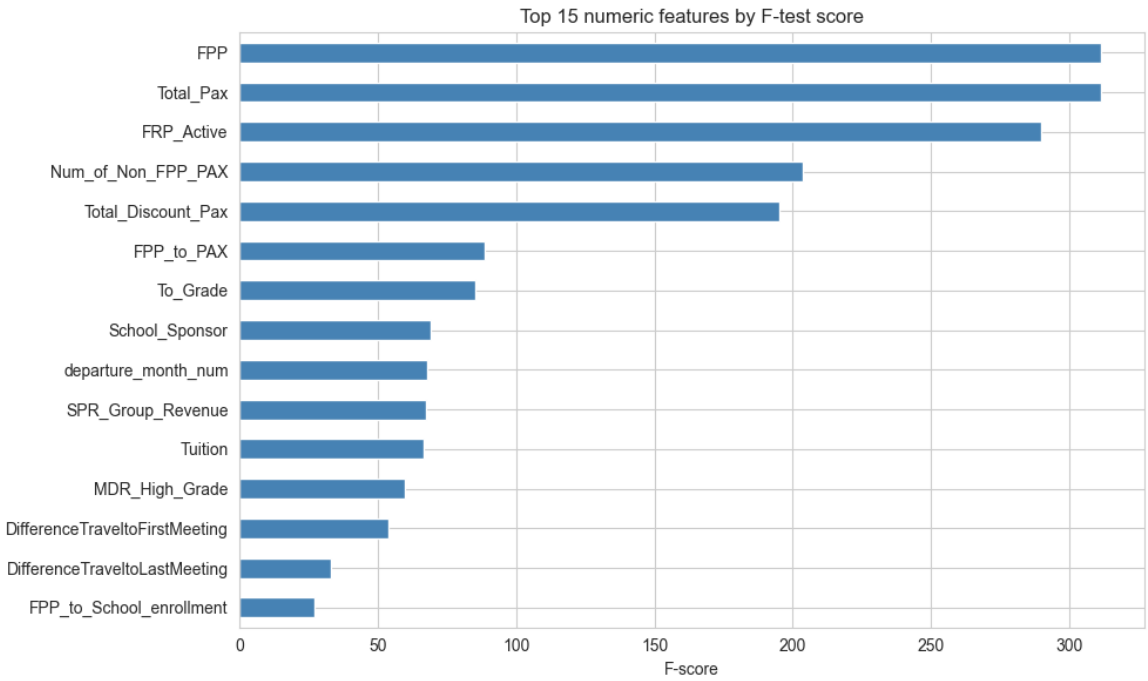
5.6 F-test ranking of numeric features

As shown in class on the breast cancer dataset, we can use the F-test (`f_classif`) to rank numeric features by how well they separate the two classes. This gives us a quantitative counterpart to the visual boxplots above.

In [18]:
```
from sklearn.feature_selection import SelectKBest, f_classif

# quick numeric-only matrix for f_classif (fills NaN with median as
X_num = df_train.select_dtypes(include='number').drop(columns='Reta
y = df_train['Retained']

selector = SelectKBest(f_classif, k='all')
```

```
selector.fit(X_num, y)

f_scores = pd.Series(selector.scores_, index=X_num.columns).sort_va
print("F-test scores (higher = more discriminating):")
print(f_scores.round(2))

fig, ax = plt.subplots(figsize=(10, 6))
f_scores.head(15).plot(kind='barh', ax=ax, color='steelblue')
ax.invert_yaxis()
ax.set_title('Top 15 numeric features by F-test score')
ax.set_xlabel('F-score')
plt.tight_layout()
plt.show()
```

```
F-test scores (higher = more discriminating):
FPP                                 311.60
Total_Pax                           311.59
FRP_Active                          289.65
Num_of_Non_FPP_PAX                  203.72
Total_Discount_Pax                  194.96
FPP_to_PAX                           88.38
To_Grade                             85.28
School_Sponsor                       68.82
departure_month_num                  67.65
SPR_Group_Revenue                    67.24
Tuition                              66.45
MDR_High_Grade                       59.73
DifferenceTraveltoFirstMeeting       53.76
DifferenceTraveltoLastMeeting        32.78
FPP_to_School_enrollment             27.18
FRP_Cancelled                        22.23
Total_School_Enrollment              19.62
From_Grade                           14.94
NumberOfMeetingswithParents          11.00
Cancelled_Pax                        10.88
Days                                  5.09
Parent_Meeting_Flag                   2.73
rpl_window_days                       2.35
deposit_lead_days                     1.79
EZ_Pay_Take_Up_Rate                   1.72
booking_lead_days                     1.29
FRP_Take_up_percent_                  0.23
MDR_Low_Grade                         0.11
SPR_New_Existing                      0.06
departure_dow                         0.00
dtype: float64
```



Takeaway: the top features by F-score are the ones that best separate retained vs churned clients. These ranks are consistent with the boxplots in 5.4 and the correlations in 5.5, which gives us a triangulated view of which predictors carry real signal. Features with very low F-scores are candidates for removal, though we'll leave the final call to the model (tree-based importance and logistic regression coefficients).

6. Final feature selection

Building on the EDA, we remove four types of columns before feeding the model:

Identifiers (`ID_SALES` , `ID_FINANCE` , `ID_CRM`) — not predictive. Raw dates — already replaced by the engineered deltas in `preprocess()` (`booking_lead_days` , `rpl_window_days` , `deposit_lead_days` , `departure_month_num`). Keeping the raw dates alongside them would be redundant. Redundant grade parts — `GroupGradeType` and `SchoolGradeType` already encode the Low + High combination; we keep the combined version. Features with near-zero signal — from section 5.6, `departure_dow` , `SPR_New_Existing` , `MDR_Low_Grade` , and `FRP_Take_up_percent_` all show F-scores below 0.25 and correlations with Retained close to zero. These are effectively noise and we drop them.

We keep the highly correlated numeric pairs (`FPP` / `Total_Pax` , passenger-count cluster) in the dataset — the model itself will handle multicollinearity via regularization (logistic regression) or feature importance (tree-based models). The test-set `ID_SALES` is kept separately so we can attach it back to the predictions at submission time.

In [19]:
```
def select_features(df):
    drop_cols = [
        # identifiers — not predictive
        'ID_SALES', 'ID_FINANCE', 'ID_CRM',
        # raw dates — replaced by engineered deltas in preprocess()
        'Departure_Date', 'Return_Date',
        'Early_RPL', 'Latest_RPL',
        'Initial_System_Date', 'Deposit_Date',
        'FirstMeeting', 'LastMeeting',
        # redundant grade parts — kept as the combined Group/School
        'GroupGradeTypeLow', 'GroupGradeTypeHigh',
        'SchoolGradeTypeLow', 'SchoolGradeTypeHigh',
        # near-zero F-score and correlation with target (see sectio
        'departure_dow', 'SPR_New_Existing',
        'MDR_Low_Grade', 'FRP_Take_up_percent_',
    ]
    return df.drop(columns=[c for c in drop_cols if c in df.columns


# keep test IDs for the submission file
test_ids = df_test['ID_SALES'].copy()

# split features from target
y_train = df_train['Retained']
X_train = select_features(df_train.drop(columns='Retained'))
X_test  = select_features(df_test)

print(f"X_train: {X_train.shape}")
print(f"X_test:  {X_test.shape}")
X_train.head(3)
```

```
X_train: (4153, 41)
X_test:  (630, 41)
```

Out[19]:

	Program_Code	From_Grade	To_Grade	Group_State	Days	Travel_Type
0	CC	10.0	10.0	CA	24	A
1	CC	5.0	6.0	CA	18	A
2	CC	7.0	7.0	OR	13	A

3 rows × 41 columns

7. Preprocessing pipeline

We now build the `ColumnTransformer` that sklearn will apply inside each CV fold. This is what guarantees **no data leakage**: imputation medians, scaler means, and one-hot categories are all learned on training folds only.

- **Numeric columns** (includes `Int8`, `float`, plain `int`): median imputation (robust to outliers) + standard scaling (needed for Logistic Regression; harmless for trees).

- **Categorical columns** (`object` and `category` dtypes): most-frequent imputation
  - one-hot encoding with `handle_unknown='ignore'` so unseen categories at prediction time don't crash the pipeline.

⚠️ **Important:** we use `select_dtypes(include='number')` and `include=['object', 'category']` because our preprocessing produces `Int8` nullable integers and `category` dtype. A naive `include=['int64', 'float64']` would silently drop many features!

In [20]:
```
# split train dataset into numeric and categorical columns
num_cols = X_train.select_dtypes(include='number').columns.tolist()
cat_cols = X_train.select_dtypes(include=['object', 'category']).co

# showcase columns
print(f"Numeric columns: {len(num_cols)}")
print(f"Categorical columns: {len(cat_cols)}")

# fill NaN with the column's median, then standardize
# we use the mean because it's less sensitive to outliers
# scaling is needed for logistic regression
numeric_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler',  StandardScaler()),
])


# fill NaN with the most common value,
categorical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_outpu
])

# ColumnTransformer applies numeric pipeline to numeric columns and
# then we concatenate the results --> single numeric matrix, for an
preprocessor = ColumnTransformer([
    ('num', numeric_transformer, num_cols),
    ('cat', categorical_transformer, cat_cols),
])
```

```
Numeric columns: 26
Categorical columns: 15
```

8. Model comparison via cross-validation

We compare three progressively more complex models:

1. **Logistic Regression** — simple linear baseline. If a complex model can't beat this, the task is essentially linear and we should keep things simple.
2. **Random Forest** — non-linear, captures interactions, robust to outliers.
3. **Gradient Boosting** — usually the strongest classical tabular model.

Evaluation uses **5-fold stratified cross-validation** (stratified preserves class balance across folds) and two metrics:

- **ROC-AUC** — captures ranking quality, threshold-independent.
- **F1** — balances precision and recall for the positive class.

In [21]:
```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random
    'Random Forest':       RandomForestClassifier(n_estimators=200,
    'Gradient Boosting':   GradientBoostingClassifier(n_estimators=
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
results = {}

for name, model in models.items():
    pipe = Pipeline([('prep', preprocessor), ('clf', model)])
    scores = cross_validate(
        pipe, X_train, y_train, cv=cv,
        scoring=['roc_auc', 'f1'],
        n_jobs=-1,
    )
    results[name] = {
        'ROC-AUC':     scores['test_roc_auc'].mean(),
        'ROC-AUC std': scores['test_roc_auc'].std(),
        'F1':          scores['test_f1'].mean(),
        'F1 std':      scores['test_f1'].std(),
    }

results_df = pd.DataFrame(results).T.round(4)
print(results_df)

best_name = results_df['ROC-AUC'].idxmax()
print(f"\nBest model by ROC-AUC: {best_name}")
```

```
                     ROC-AUC  ROC-AUC std      F1  F1 std
Logistic Regression   0.8066       0.0145  0.7981  0.0099
Random Forest         0.9239       0.0122  0.8874  0.0097
Gradient Boosting     0.8492       0.0124  0.8274  0.0088

Best model by ROC-AUC: Random Forest
```

8.1 Sanity Check

ROC-AUC and F1 give aggregate scores but don't show what kinds of mistakes the model makes. A confusion matrix, computed out-of-fold via cross-validation, shows how often retained clients are mistaken for churned and vice versa. This is what matters for the business.

In [22]:
```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import confusion_matrix, classification_report

# refit the best pipeline and get out-of-fold predictions
```

```
best_pipe = Pipeline([('prep', preprocessor), ('clf', models[best_n
oof_preds = cross_val_predict(best_pipe, X_train, y_train, cv=cv, n

# print the confusion matrix
cm = confusion_matrix(y_train, oof_preds)
print("Confusion matrix (rows = true, columns = predicted):")
print(pd.DataFrame(cm,
                   index=['Churned (0)', 'Retained (1)'],
                   columns=['Pred Churned', 'Pred Retained']))
```

```
Confusion matrix (rows = true, columns = predicted):
              Pred Churned  Pred Retained
Churned (0)           1192            439
Retained (1)           160           2362
```

Takeaway:

- Top-left: 1192 --> customers who actually churned, and the model correctly identified them as churned.
- Top-right: 439 --> customers who actually churned, but the model mistakenly identified them as retained.
- Bottom-left: 160 --> customers who actually retained, but the model wrongly said they would churn.
- Bottom-right: 2362 --> customers who actually retained, and the model correctly identified them as retained.

This means that out of 4153 total customers, the model got 3554 (1192 + 2362) right, that is about 86% overall accuracy.

In [23]:
```
# classification report
print("\nClassification report:")
print(classification_report(y_train, oof_preds, target_names=['Chur
```

```
Classification report:
              precision    recall  f1-score   support

     Churned       0.88      0.73      0.80      1631
    Retained       0.84      0.94      0.89      2522

    accuracy                           0.86      4153
   macro avg       0.86      0.83      0.84      4153
weighted avg       0.86      0.86      0.85      4153
```

Takeaway:

- The model has overall 86% accuracy, which we consider pretty solid.
- The model is great at spotting retained customers: 2362 ÷ 2522 = 94% correct on the retained class.
- We consider the model to be okay at spotting customers who will leave (73% caught). However, this means that for every 100 customers who are about to churn, the model flags only 73 of them, and 27 go unnoticed.

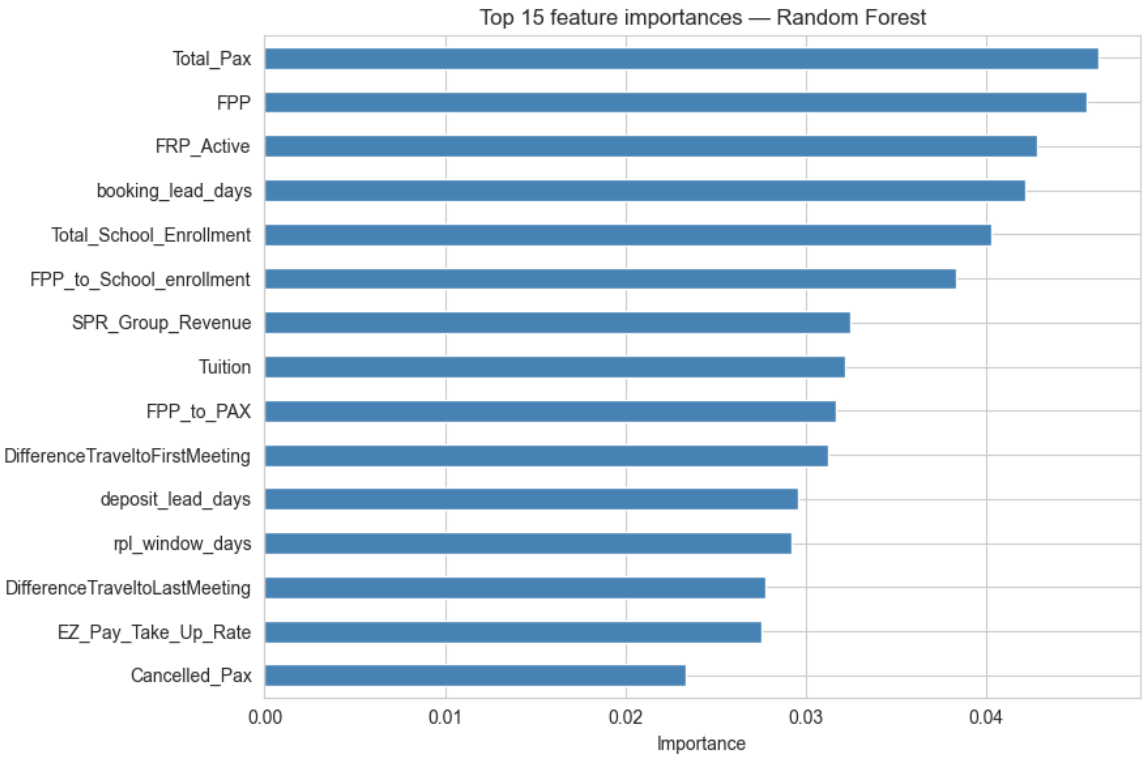
9. Final model — fit on all training data

Now that we've picked the winner, we refit it on the **full training set** (no hold-out anymore — we want every labelled row before predicting on test).

We also extract **feature importances** from the tree-based model. This is what the business team actually cares about: it tells us *why* churning customers churn.

In [24]:
```
best_pipe = Pipeline([('prep', preprocessor), ('clf', models[best_n
best_pipe.fit(X_train, y_train)

# Extract post-one-hot feature names so importances are labelled co
if hasattr(models[best_name], 'feature_importances_'):
    ohe_cats = (
        best_pipe.named_steps['prep']
                 .named_transformers_['cat']
                 .named_steps['encoder']
                 .get_feature_names_out(cat_cols).tolist()
    )
    feature_names = num_cols + ohe_cats

    importances = pd.Series(
        best_pipe.named_steps['clf'].feature_importances_,
        index=feature_names
    ).sort_values(ascending=False)

    print("Top 15 features:")
    print(importances.head(15).round(4))

    fig, ax = plt.subplots(figsize=(9, 6))
    importances.head(15).sort_values().plot(kind='barh', ax=ax, col
    ax.set_title(f'Top 15 feature importances — {best_name}')
    ax.set_xlabel('Importance')
    plt.tight_layout()
    plt.show()
```

```
Top 15 features:
Total_Pax                          0.0462
FPP                                0.0456
FRP_Active                         0.0428
booking_lead_days                  0.0421
Total_School_Enrollment            0.0402
FPP_to_School_enrollment           0.0383
SPR_Group_Revenue                  0.0324
Tuition                            0.0322
FPP_to_PAX                         0.0317
DifferenceTraveltoFirstMeeting     0.0313
deposit_lead_days                  0.0296
rpl_window_days                    0.0292
DifferenceTraveltoLastMeeting      0.0278
EZ_Pay_Take_Up_Rate                0.0276
Cancelled_Pax                      0.0233
dtype: float64
```



One relevant insight is that three themes primarily drive churn: group size, how early the school engages with the agency, and the financial commitment they make. Larger groups, earlier bookings, and higher financial investment are associated with retention.

10. Generating predictions for the test set

Final step: apply the trained pipeline to the test set and save predictions in the format required by the assignment (`ID_SALES, Retained`).

In [25]:
```
test_preds = best_pipe.predict(X_test)

submission = pd.DataFrame({
    'ID_SALES': test_ids,
```

```
    'Retained': test_preds,
})

submission.to_csv('predictions.csv', index=False)

print(f"Saved {len(submission)} predictions to predictions.csv")
print("\nPrediction distribution:")
print(submission['Retained'].value_counts())
print("\nFirst 10 predictions:")
print(submission.head(10))
```

```
Saved 630 predictions to predictions.csv

Prediction distribution:
Retained
1    409
0    221
Name: count, dtype: int64

First 10 predictions:
  ID_SALES  Retained
0  CC1040A         1
1  CC1525A         1
2  CC2011A         1
3  CD1132A         1
4  CD1178T         1
5  CD1328A         0
6  CD1391A         0
7  CD1422T         0
8  CD1433A         0
9  CD1539A         1
```

11. Business takeaways

This section drives the presentation's narrative. Based on the feature importances, we can profile churn risk and suggest concrete retention levers:

- **Time-related features** (`booking_lead_days` , `rpl_window_days` , `deposit_lead_days`) suggest that **how early and for how long a client engages** with the agency matters. Agents could actively nurture clients with short RPL windows — they're at higher risk.
- **Finance features** (`FRP_Take_up_percent_` , `EZ_Pay_Take_Up_Rate` , `Tuition`) reflect the client's financial commitment. Low take-up on insurance or direct debit may indicate weaker attachment to the service.
- **CRM engagement** (`NumberOfMeetingswithParents` , `Parent_Meeting_Flag` , `has_crm_data`) suggests **human contact matters**. Schools with more parent meetings are more likely to return.
- **Operational features** (`cancellation_rate` , `Days`) capture the trip experience — high cancellation rates or outlier durations may correlate with churn.

Recommended actions for Yeti Travel

1. **Early-warning dashboard** — flag accounts whose predicted churn probability exceeds a threshold and route them to a dedicated retention agent.
2. **Parent meetings as a retention lever** — proactively promote meetings for schools with zero recorded meetings.
3. **Engagement-depth policy** — encourage early booking and full-duration RPL campaigns through time-limited incentives.
4. **Customer satisfaction tracking** — as Angela originally hinted, the biggest blind spot is the lack of direct satisfaction data. A simple post-trip survey would add powerful signal to future iterations of the model.